

EE147 Practice Exam 2 — Answer Key with Reasoning

Kaushik Vada

Q1. Tiled SGEMM Launch (4 pts)

1024×1024 output, `TILE_WIDTH = 32`, so block = $32 \times 32 = 1024$ threads.

a) **Answer:** 32 warps per block.

Why: $1024 \text{ threads} / 32 \text{ threads-per-warp} = 32 \text{ warps}$.

b) **Answer:** 1024 blocks in the grid.

Why: Grid is $\lceil 1024/32 \rceil \times \lceil 1024/32 \rceil = 32 \times 32 = 1024$.

Q2. Histogram stride (4 pts)

Answer: `int stride = blockDim.x * gridDim.x;`

Why: This is the total number of threads in the grid. With each thread starting at `threadIdx.x + blockIdx.x*blockDim.x` and stepping by the grid size, adjacent threads hit adjacent input bytes in each iteration — **interleaved** (coalesced) partitioning. A sectioned scheme (`stride = 1`, `length = total/threads`) would not be coalesced.

Q3. Why shared-memory privatization helps (4 pts)

Answer: Global atomics on a hot bin serialize through long-latency DRAM/L2 accesses ($\sim 1000+$ cycles each), so heavy contention drops throughput to roughly $1/\text{latency}$. Privatization gives each block its own private histogram in shared memory. Shared-memory atomics have much shorter latency and only contend with threads in the same block. After the block builds its private bins, it merges into the global histogram with one atomic per bin, drastically reducing total contention and serialization. Typical speedups are $>10\times$.

Q4. Unified Memory (13 pts)

a) **Pascal+ behavior on concurrent CPU/GPU access.**

Answer: The CPU access page-faults; the runtime migrates the page back to the CPU on demand and the write succeeds. No segmentation fault.

Why: Pascal+ supports concurrent CPU/GPU access to managed memory via demand paging.

b) **Pre-Pascal behavior.**

Answer: Fatal segmentation fault on the CPU side. The CPU cannot touch managed memory while the kernel is running; you must call `cudaDeviceSynchronize()` first.

Why: Pre-Pascal UM moves managed data en-masse at kernel launch, and the CPU mapping is removed until the kernel returns and the runtime resolves.

c) 64 GB allocation on 16 GB Pascal GPU.

Answer: Yes, it succeeds (with appropriate `size_t`). Pascal+ supports oversubscription — pages are migrated to GPU on demand and evicted as needed. It would *fail* on Kepler/Maxwell.

d) Read-mostly hint.

Answer: `cudaMemAdvise(ptr, count, cudaMemAdviseSetReadMostly, device);`

Why: The runtime keeps a local read-only copy on each processor that touches it (read duplication). A write by any processor collapses all copies into one, after which subsequent reads fault and re-duplicate. The `device` argument is ignored for this hint.

Q5. Tiled SGEMM Analysis (12 pts)

a) Block = $16 \times 16 = 256$ threads. Each thread loads 1 M and 1 N element.

Answer: $2 \cdot 256 = 512$ global loads per block per phase.

b) Phases = Width/TILE_WIDTH = $1024/16 =$

Answer: 64 phases.

c) Two tile arrays, each 32×32 floats = 4096 bytes. Total

Answer: 8192 bytes = 8 KB per block.

Why: $2 \cdot 32 \cdot 32 \cdot 4 = 8192$.

d) $\lfloor 48,000/8192 \rfloor =$

Answer: 5 blocks

co-resident based on the shared-memory limit alone (real occupancy may be further constrained by register usage and the threads-per-SM cap).

Q6. MC — which API actually moves data? (3 pts)

Answer: `cudaMemPrefetchAsync(...)`.

Why: The three `cudaMemAdvise` variants are hints — they do not by themselves move data. Only `cudaMemPrefetchAsync` actively migrates pages to the indicated destination.

Q7. MC — coalesced access pattern (3 pts)

Answer: `expr = (something independent of threadIdx.x) + threadIdx.x`.

Why: For all threads in a warp to hit consecutive addresses (same burst section), the address must change by exactly 1 as `threadIdx.x` changes by 1. The other forms either don't involve `threadIdx.x`, are degenerate, or scale by stride.

Q8. MC — MPS with 5 streams per process (3 pts)

Answer: False dependencies/serialization due to more streams per client than channels.

Why: An MPS client gets 2 work queues per channel — with 5 streams squeezed in, the extra streams share queues and incur false serialization. MPS does not auto-distribute across GPUs. Kernels from different processes *do* overlap under MPS — that’s the whole point — so the “always serialize” option is wrong.

Q9. MC — DP launches per block (3 pts)

Answer: 256.

Why: Device-side launches are **per-thread**: every thread that executes the launch line spawns a child grid. To launch once per block, guard with `if (threadIdx.x == 0)`.

Q10–Q15 True / False (12 pts)

Q10. “`atomicAdd` returns the new value.”

Answer: F

— it returns the *old* value (the read before the add).

Q11. “Coalescing rules apply to shared memory the same way as global memory.”

Answer: F

— shared memory has its own access model based on banks/bank conflicts, not DRAM burst sections.

Q12. “The two `__syncthreads()` in tiled SGEMM can be removed for large block sizes.”

Answer: F

— both barriers are required regardless of block size. The first ensures the tile is fully loaded; the second ensures it’s fully consumed before the next phase overwrites it.

Q13. “Device-side `cudaDeviceSynchronize()` implies `__syncthreads()`.”

Answer: F

— they’re independent. The lecture explicitly states `cudaDeviceSynchronize()` does *not* act as a block-wide thread barrier.

Q14. “P2P copy between GPUs on different IOHs still completes correctly.”

Answer: T

— the copy works, but it gets staged through host memory because QPI between IOHs isn’t compatible with PCIe P2P. Slower, but correct.

Q15. “`cudaMemAdviseSetAccessedBy` immediately moves data.”

Answer: F

— it sets up a (P2P) mapping so the indicated processor can access without faulting, but data does not move and its location is unchanged.

Q16. Boundary handling for rectangular matmul (6 pts)

M is $j \times k$, N is $k \times l$, output P is $j \times l$. With $j = 1000$, $k = 800$, $l = 1500$, `TILE_WIDTH = 16`.

- a) `gridDim.x =  $\lceil l/16 \rceil = \lceil 1500/16 \rceil = 94$ .`
`gridDim.y =  $\lceil j/16 \rceil = \lceil 1000/16 \rceil = 63$ .`

Answer: `x = 94, y = 63`.

- b) Boundary for loading the M tile (rows from M are bounded by j , columns by k):

Answer: `(Row < j) && (p * TILE_WIDTH + tx < k)`

If false, write 0 into shared memory instead of loading.

- c) Boundary for writing the output P (bounded by j rows, l columns):

Answer: `(Row < j) && (Col < l)`

Q17. Dynamic Parallelism trace (12 pts)

- a) **Answer:** 4 child grids.

Why: Launches are per-thread; only threads with `tid < 4` hit the launch line, so threads 0, 1, 2, 3 each spawn one child grid.

- b) **Answer:** All 4 child grids have completed at `PC = C`, regardless of which thread you're asking about.

Why: `cudaDeviceSynchronize()` on the device is per-block: it waits for every launch made by any thread in the block. Thread 5 didn't launch anything, but it still sees all 4 children finished after this sync.

- c) **Answer:** Yes, with one caveat about visibility.

Why: By `PC = D`, every child grid launched by every thread has finished due to the device-sync. Their writes to global memory are completed. However, since `cudaDeviceSynchronize()` does *not* imply `__syncthreads()`, the textbook-safe pattern adds a `__syncthreads()` after the device-sync if you want all threads in the block to see consistent state before reading. Without that extra barrier, ordering relative to other threads' control flow isn't guaranteed, though the child writes themselves have landed.

- d) **Answer:** No.

Why: `__syncthreads()` is a thread-block barrier — it doesn't wait for any running child kernels. The parent would race ahead and read `out[tid]` before the children's writes are guaranteed to be visible. You need the device sync for that.

Q18. Shared-memory atomic semantics (4 pts)

`histo_private[3]` starts at 7. Two threads do `atomicAdd(&histo_private[3], 1)` in some order; the hardware serializes them.

Answer: Final value of `histo_private[3] = 9`. One thread receives 7 as the return (the *old* value), and the other receives 8. Which thread sees which depends on hardware scheduling — you cannot tell from the source.

Reflection notes after grading yourself

- If you missed Q4: rebuild the UM mental model (Pascal+ vs pre-Pascal, hints vs prefetch).
- If you missed Q5: re-derive the FLOP/load ratio and shared-memory footprint for the tile sizes — likely to appear in multiple places.
- If you missed Q17: read the DP rules in the study guide again. Launch per-thread, sync per-block, and `cudaDeviceSynchronize()` $\neq$ `__syncthreads()` is the single most testable fact about DP.
- If you missed any T/F: those are pure memorization. They're worth re-drilling with the flashcard deck the morning of the exam.